

VARP K26-Memory FPGA

발표·논문 방어 스테디팩

한국어 학습자료 · Q&A; 89문항

한 장 요약

연구 질문

정적 큐 소유권으로 생기는 불균형에서, locality-aware Work Stealing이 원격 이동 비용을 통제하면서 p95/p99 tail latency를 줄일 수 있는가?

구조

K26 측에는 4개 compute cluster와 signed-INT8 16×4 MatVec 경로가 있다. 외부 Memory FPGA 후보에는 4-channel DDR3L command plane과 4-bundle link routing plane이 있다. 하지만 DDR response→link receive→payload store→MatVec의 닫힌 end-to-end 경로는 아직 없다.

핵심 결과

- graph-derived: Gemma 3 1B ONNX 7,837 nodes, token당 183 projections.
- RTL-simulated: 실제 weight tile 3/3 INT32 parity, synthetic steal→MatVec 3 accepted=3 completed.
- analytical: full-overlap에서 S3는 S1보다 skew p95 18.12%, mixed p95 17.59% 감소.
- analytical: S3는 S2보다 remote weight bytes를 skew 37.84%, mixed 22.16% 감소.
- capacity model: INT8 context-32K 2.4301 GiB는 명목 4 GB에도 들어간다.

결론

S3는 불균형 조건의 후보 정책이다. 외부 8 GiB의 채택 이유는 용량이 아니라 향후 측정할 로컬 대역폭·경합·전력에 달려 있다. 보드 성능·전력, 완전한 3B 실행, 제작 준비 상태는 주장하지 않는다.

개념 지도

Gemma ONNX hash

- └─ graph inventory (7,837 nodes)
- └─ projection ledger (183/token)
- └─ representative INT8 tile → RTL MatVec parity
- └─ analytical TileJob stream → S0/S1/S2/S3 비교
- └─ p95/p99
- └─ completion time
- └─ remote bytes

K26 compute plane — runnable RTL

Memory command plane — independent command output

Link routing plane — independent bundle output

DDR response/DMA/receive/CDC — BLOCKED integration gate

KiCad native source → bounded ERC/DRC → NOT FOR FABRICATION

핵심은 화살표 하나가 곧 증거 결합을 뜻하지 않는다는 점이다. 각 연결은 실제 코드·테스트·로그가 있을 때만 닫힌 것으로 취급한다.

논문 순회

I-II. 문제와 차별점

'메모리가 병목'이라는 일반론을 TileJob 소유권, queue skew, locality cost, remote-byte 회계로 구체화한다. 알고리즘 신규성이나 최고 성능을 주장하지 않고 evidence chain을 기여로 둔다.

III. 실제 workload

해시로 묶인 Gemma 3 1B ONNX에서 7,837-node inventory와 token당 183 projection을 얻는다. 실제 weight 16×4 tile 세 개만 RTL MatVec parity를 확인했으므로 model-wide accuracy로 확대하면 안 된다.

IV-V. 구조와 정책

S0는 이상적 중앙 FIFO, S1은 static local, S2는 oldest eligible stealing, S3는 age에서 locality penalty를 뺀 score를 사용한다. compute/memory/link plane의 독립성을 명시한다.

VI-VII. 방법과 결과

동일한 1,000-job synthetic streams와 seed 5개를 비교한다. full-overlap과 sequential은 물리 timing이 아니라 service boundary다. S3는 skew/mixed tail과 remote bytes에 이득이 있지만 balanced/hotspot에는 이득이 없고 completion time은 소폭 악화될 수 있다.

VIII. 물리 범위와 결론

KiCad coupon은 native source와 제한된 검사 증거다. 55 unrouted nets, SI/PI/PDN/thermal 미검증 때문에 제작 준비 상태가 아니다. 다음 단계는 response가 있는 DMA/link/DDR loop와 보드 계측이다.

슬라이드 순회

- 1. 연구 질문과 analytical/model/hybrid 한정부터 밝힌다.
- 2. 병목 일반론을 정책·증거 문제로 바꾼 이유를 말한다.
- 3. compute, memory command, link plane을 분리한다.
- 4. graph→TileJob→MatVec 중 실제로 닫힌 구간을 짚는다.
- 5. S0/S1/S2/S3 trade-off를 비교한다.
- 6. imbalance→victim search→score→steal→exact-once를 설명한다.
- 7. graph-derived ledger와 host ORT·analytical model을 분리한다.
- 8. p95/p99, compute duty, reservation occupancy, remote bytes를 함께 읽는다.
- 9. energy/cost는 추정이며 board power·전체 BOM이 아님을 말한다.
- 10. KiCad coupon의 bounded pass와 fabrication blocker를 함께 보여준다.
- 11. proven/modeled/not claimed를 한 문장씩 말한다.
- 12. 다음 검증 loop와 Q&A 질문으로 닫는다.

용어집

용어 · 뜻

Prefill · 입력 문맥 전체를 병렬적으로 처리해 KV cache를 만드는 단계

Decode · 이전 KV를 재사용하며 다음 토큰을 한 개씩 생성하는 단계

Projection · attention/MLP의 선형변환; decode에서 가중치 재읽기 비중이 큼

MatVec · 행렬과 벡터의 곱; 이 artifact는 signed INT8 16×4 primitive를 사용

TileJob · 일부 K/N 범위와 주소·소유권·친화도를 가진 작업 단위

FCFS · 도착 순서 우선 처리

Work Stealing · 빈 worker가 다른 queue의 eligible job을 가져오는 정책

Locality cost · 원격 weight/activation/reduction/bundle mismatch의 모델 비용

p95/p99 · 지연 표본의 95/99 백분위; tail을 나타냄

Compute duty · 실제 compute cycle 비율

Reservation occupancy · dispatch 후 대기까지 포함한 cluster 예약 비율

Backpressure · downstream이 받을 수 없어 upstream 전송을 늦추는 흐름 제어

CDC · 서로 다른 clock domain 사이의 안전한 신호 전달

DDR channel/bank/row · 독립 채널, 내부 병렬 bank, bank 안의 row 주소 계층

DRAMsim3 · DRAM timing simulator; 본 repo의 snapshot은 replay와 결합되지 않음

SpinalHDL · Scala 기반 하드웨어 기술 언어

Verilator · HDL을 C++ 모델로 변환해 시뮬레이션하는 도구

ERC/DRC · 전기 규칙/배선 규칙 검사; 제작 가능성 전체를 보장하지 않음

구조와 데이터 흐름

닫힌 compute path

MatVecTileCommand → TileScheduler → associative payload store → ComputeClusterArray → MatVecTileResult

이 경로는 runnable RTL이며 synthetic steal identity와 actual MatVec result를 연결한다.

독립 memory/link planes

memoryRequest → channel mapping → bank-aware queue → memoryCommands

linkInput → bundle select → linkBundles

둘 다 외부 입력에서 시작한다. response interface, DMA sequencing, GT wrapper, receive path, returned weight insertion은 없다.

FIFO를 구분하는 이유

TileJob queue는 '누가 실행하는가'를 정한다. Transport FIFO는 'byte가 언제 이동하는가'를 정한다. 합치면 queue imbalance와 link backpressure를 구분할 수 없다.

실험과 지표

Synthetic workload는 balanced, skew, hotspot, bursty, mixed 각각 1,000 job이고 seed는 19/23/29/31/43이다. 동일 seed의 3회 반복은 결정성 검사이며 표본 수를 늘리지 않는다.

Full-overlap은 $\text{data-ready} = \max(\text{link-end}, \text{memory-end})$, sequential은 두 서비스를 더한다. 두 모델 모두 DMA/PHY cycle timing이 아니다.

지표는 p50/p95/p99, completion time, successful steals, remote weight bytes, compute duty, reservation occupancy, unreserved idle을 함께 본다. occupancy가 99%여도 MAC이 99% 계산했다는 뜻은 아니다.

정상성 조건은 input/dispatched/completed ID set 동일, duplicate 0, timeout false다.

전력·비용·에너지

에너지는 측정값이 아니다. INT8 MAC 1/5/15 pJ와 link 24/51.2/120 pJ/bit 민감도를 사용하고, DRAM command dynamic 항을 더한다. Refresh, idle, controller, PHY, regulator와 보드 전체 전력은 제외한다.

비용은 DRAM package와 package 내부 physical die만 정규화한다. K26, Memory FPGA, PCB, 조립, 전원과 커넥터는 제외된다. 따라서 '전체 시스템 가격'이라고 부르면 안 된다.

외부 8 GiB는 Gemma 1B capacity 때문에 필수라는 결과가 아니다. context-32K INT8 모델 2.4301 GiB가 명목 4 GB에 들어가므로, 채택은 대역폭·경합·전력 측정 뒤에 결정한다.

KiCad와 물리 범위

Native KiCad source와 validation coupon은 실제 산출물이다. ERC/제한 DRC는 선언된 규칙과 현재 범위 안에서의 검사다.

그러나 55 unrouted nets, DDR topology/length matching 미완료, SI/PI/PDN/thermal/EMC 미검증, FPGA pin planning·MIG·GT·clock closure 부재 때문에 NOT FOR FABRICATION이다.

다음 물리 검증 순서는 schematic/net class 확정 → routing/length match → ERC/DRC 0 → SI/PI/PDN → FPGA constraints/timing → 제작 → bring-up → calibrated power/bandwidth다.

주장 경계

직접 또는 제한적으로 지지

- ONNX hash와 graph inventory
- token당 projection ledger
- representative tile 3/3 RTL parity
- synthetic steal→MatVec exact-once
- native KiCad source와 bounded checks

모델·추정

- S0-S3 p95/p99/completion/remote bytes
- hybrid token latency
- dynamic energy sensitivity
- memory-die-cost normalization과 capacity

주장하지 않음

- 완성된 K26-Memory FPGA accelerator
- 보드 성능·대역폭·전력 측정
- full 3B 실행
- 닫힌 DDR response/link receive/MatVec payload loop
- fabrication-ready PCB 또는 SI/PI/thermal signoff

Q&A Bank

총 89문항. 답변 시 증거 유형을 먼저 말한다.

Q01. 연구의 한 문장 질문은?

- 짧은 답: 불균형 조건에서 locality-aware stealing이 이동 비용을 통제하며 tail을 줄이는지 묻는다.
- 상세 답: 정적 home queue가 만든 idle과 긴 tail을 대상으로 S1과 S3를 동일 ledger에서 비교한다.
- 증거 포인터: paper/final/submission_manuscript.md §I, §VII-A
- 과장 위험: '외부 FPGA가 빠르다'로 바꾸면 보드 증거를 과장한다.

Q02. 왜 단순히 '메모리 대역폭이 중요하다'는 말보다 진전인가?

- 짧은 답: 병목을 queue policy와 검증 가능한 지표로 바꿨기 때문이다.
- 상세 답: 실제 graph-derived job, p95/p99, remote bytes, exact-once 조건과 다음 물리 관문을 연결했다.
- 증거 포인터: paper/final/submission_manuscript.md §I-II
- 과장 위험: 알고리즘 신규성이나 최고 성능으로 과장하면 안 된다.

Q03. 핵심 기여 세 가지는?

- 짧은 답: 실제 graph ledger, 정책 감사, evidence chain이다.
- 상세 답: ONNX projection 순서, 동일 stream의 tail/traffic 비교, 제한 RTL·KiCad와 claim boundary의 연결이다.
- 증거 포인터: paper/final/submission_manuscript.md §I
- 과장 위험: 완성 시스템 구현을 기여로 추가하면 과장이다.

Q04. 이 연구는 완성된 accelerator인가?

- 짧은 답: 아니다.
- 상세 답: Compute RTL은 달렸지만 DDR response, DMA, GT/receive, returned weight insertion이 없다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 세 plane의 존재를 end-to-end 통합으로 오해하면 안 된다.

Q05. 어떤 증거 유형이 있는가?

- 짧은 답: Direct, RTL-simulated, graph-derived, modeled, blocked다.
- 상세 답: 각 유형은 지원 가능한 주장과 금지 주장을 표 3에서 고정한다.
- 증거 포인터: docs/evidence.md; paper §IV 표 3
- 과장 위험: 유형을 생략하면 분석값이 실측처럼 들린다.

Q06. 결론을 한 문장으로 말하면?

- 짧은 답: S3는 불균형 조건의 후보이며 외부 memory 채택은 실측 baseline까지 보류한다.
- 상세 답: Skew/mixed tail/traffic 이득은 analytical이며 capacity만으로 8 GiB 필요성을 지지하지 않는다.
- 증거 포인터: paper §VII-IX
- 과장 위험: 'S3 채택 확정'으로 말하지 않는다.

Q07. 왜 evidence boundary가 연구 기여인가?

- 짧은 답: 재현 가능한 범위만 주장하게 만들기 때문이다.
- 상세 답: Graph, RTL, model, physical proposal의 연결 여부를 명시해 다음 실험을 선택할 수 있다.

- 증거 포인터: docs/evidence.md
- 과장 위험: 한계를 단순한 미완성 변명으로 축소하지 않는다.

Q08. 가장 중요한 다음 실험은?

- 짧은 답: DDR/link response에서 MatVec까지 payload loop를 닫는 것이다.
- 상세 답: DMA, response, credit/CDC, receive FIFO와 exact-once completion을 먼저 검증한다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 바로 보드 성능 수치를 약속하지 않는다.

Q09. Prefill이란?

- 짧은 답: 입력 문맥을 처리해 KV cache를 만드는 단계다.
- 상세 답: 여러 token을 병렬 계산할 여지가 크고 attention의 행렬 연산 형태가 decode와 다르다.
- 증거 포인터: study/04_glossary.md
- 과장 위험: 본 결과를 prefill 성능으로 확대하지 않는다.

Q10. Decode란?

- 짧은 답: 이전 KV를 사용해 다음 token을 순차 생성하는 단계다.
- 상세 답: 작은 batch에서는 projection weight를 반복 스트리밍해 memory/queue tail이 중요해진다.
- 증거 포인터: paper §I, §III
- 과장 위험: decode-32와 context-32K를 혼동하지 않는다.

Q11. Decode-32와 context-32K의 차이는?

- 짧은 답: 32 생성 token과 KV 길이 32,768의 차이이다.
- 상세 답: Decode-32는 $183 \times 32 = 5,856$ jobs이고 context-32K는 capacity 산술 조건이다.
- 증거 포인터: paper §III-A, §VI-B
- 과장 위험: 둘을 같은 workload 길이로 말하면 계산 의미가 바뀐다.

Q12. Projection workload는 무엇인가?

- 짧은 답: Attention과 MLP의 선형변환 작업이다.
- 상세 답: q/k/v/o 104, gate/up/down 78, lm_head 1로 token당 183개다.
- 증거 포인터: paper §III-A
- 과장 위험: 전체 ONNX node가 모두 projection이라고 말하지 않는다.

Q13. MatVec는 무엇인가?

- 짧은 답: 행렬 tile과 vector의 곱이다.
- 상세 답: 현재 RTL primitive는 signed INT8 16×4 입력을 INT32 누산 결과로 계산한다.
- 증거 포인터: paper §III-B; RTL sources
- 과장 위험: full projection engine이나 model execution으로 확대하지 않는다.

Q14. Tiling이 필요한 이유는?

- 짧은 답: 큰 projection을 제한된 compute/memory 단위로 나누기 위해서다.
- 상세 답: TileJob이 K/N 범위, 주소와 preferred channel/bundle을 보존한다.
- 증거 포인터: paper §IV

- 과장 위험: 현재 tile fixture가 전체 layout을 검증했다고 말하지 않는다.

Q15. INT8과 INT4의 차이는?

- 짧은 답: 가중치 비트폭과 표현 범위가 다르다.
- 상세 답: INT4는 저장·traffic을 더 줄일 수 있지만 packing, scale, accuracy와 RTL datapath 검증이 별도 필요하다.
- 증거 포인터: paper §III-B; study/04_glossary.md
- 과장 위험: 이 artifact가 INT4를 구현했다고 말하지 않는다.

Q16. 실제 tile parity 3/3은 무엇을 증명하는가?

- 짧은 답: 세 실제 16×4 weight tile의 RTL 산술 일치를 증명한다.
- 상세 답: External-data offset에서 읽고 per-tile symmetric INT8로 양자화한 값이 INT32 reference와 일치했다.
- 증거 포인터: paper §III-B
- 과장 위험: model-wide accuracy나 K26 timing을 증명하지 않는다.

Q17. ONNX graph node 수는?

- 짧은 답: 7,837개다.
- 상세 답: 해시로 고정된 artifact의 protobuf graph inventory에서 얻은 graph-derived 수치다.
- 증거 포인터: paper §III-A
- 과장 위험: 다른 모델 revision에 일반화하지 않는다.

Q18. Token당 projection 수는?

- 짧은 답: 183개다.
- 상세 답: Attention 104, MLP 78, lm_head 1의 합이다.
- 증거 포인터: paper §III-A
- 과장 위험: 실제 accelerator가 183개를 실행했다는 뜻은 아니다.

Q19. 모델 weight가 repository에 있는가?

- 짧은 답: 없다.
- 상세 답: Manifest와 hash/acquisition guide만 공개하며 weights는 release에서도 제외한다.
- 증거 포인터: models/ACQUISITION.md; models/LICENSE_NOTES.md
- 과장 위험: 재배포 권한이 있다고 암시하지 않는다.

Q20. Y700 408.445 ms는 무엇인가?

- 짧은 답: Android CPU EP의 제한된 기능 참조다.
- 상세 답: 세 번의 single-token 조건 평균이며 decode-32 RTL/DRAMsim3/accelerator timing이 아니다.
- 증거 포인터: paper §III-A
- 과장 위험: 가속기 latency로 인용하지 않는다.

Q21. TileJob의 핵심 필드는?

- 짧은 답: Identity, 범위, 주소, locality와 ownership 정보다.
- 상세 답: Job ID, arrival, layer/op, activation/weight/output address, K/N, preferred channel/bundle, priority, stealable flag를 가진다.
- 증거 포인터: paper §IV

- 과장 위험: Python과 RTL ownership 정의가 완전히 같다고 말하지 않는다.

Q22. Compute plane의 닫힌 경로는?

- 짧은 답: Scheduler→payload store→ComputeClusterArray→result다.
- 상세 답: MatVecTileCommand가 TileJob과 payload를 연결해 실제 MatVec result를 반환한다.
- 증거 포인터: docs/architecture.md
- 과장 위험: DDR에서 weight를 읽는 경로까지 닫혔다고 말하지 않는다.

Q23. Memory command plane은 무엇을 하는가?

- 짧은 답: 외부 request를 channel/bank queue를 거쳐 command로 낸다.
- 상세 답: Response interface가 없어 compute payload로 되돌아오지 않는다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 완전한 DDR controller나 DMA로 부르지 않는다.

Q24. Link plane은 무엇을 하는가?

- 짧은 답: 외부 input을 bundle로 routing한다.
- 상세 답: GT wrapper와 receive path가 없고 compute payload와 연결되지 않았다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 실제 payload bandwidth를 측정했다고 말하지 않는다.

Q25. 왜 TileJob queue와 transport FIFO를 분리하는가?

- 짧은 답: Scheduling과 byte movement의 원인을 분리하기 위해서다.
- 상세 답: 전자는 실행 소유권, 후자는 credit/backpressure와 전송 시점을 결정한다.
- 증거 포인터: paper §IV
- 과장 위험: Queue 개선을 link 개선으로 등치하지 않는다.

Q26. Backpressure란?

- 짧은 답: Downstream 수용 불가가 upstream 전송을 늦추는 현상이다.
- 상세 답: Credit/FIFO full과 소비 속도 때문에 생기며 job queue imbalance와 다른 원인이다.
- 증거 포인터: study/04_glossary.md
- 과장 위험: 현재 link plane에 완전한 backpressure가 구현됐다고 말하지 않는다.

Q27. CDC란?

- 짧은 답: 서로 다른 clock domain 사이의 안전한 전달이다.
- 상세 답: Synchronizer나 async FIFO와 metastability 분석이 필요하다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 단순 FIFO를 CDC closure로 간주하지 않는다.

Q28. DDR channel/bank/row의 관계는?

- 짧은 답: Channel은 독립 인터페이스, bank는 내부 병렬 단위, row는 bank의 열린 행이다.
- 상세 답: Mapping과 row hit가 service time과 contention을 바꾼다.
- 증거 포인터: paper §IV; study/04_glossary.md
- 과장 위험: Pin-rate를 effective bandwidth로 말하지 않는다.

Q29. 4-channel pin-rate 상한은?

- 짧은 답: 가정상 6.4 GB/s다.
- 상세 답: $4 \times 16 \text{ bit} \times 800 \text{ MT/s} \div 8$ 의 산술 상한이며 overhead 전 값이다.
- 증거 포인터: paper §IV
- 과장 위험: 실측 payload bandwidth로 인용하지 않는다.

Q30. FIFO란?

- 짧은 답: 먼저 들어온 항목을 먼저 처리하는 queue다.
- 상세 답: S1 local FCFS는 locality를 보존하지만 skew에서 idle을 만들 수 있다.
- 증거 포인터: paper §V
- 과장 위험: Central FIFO의 물리 비용이 0이라고 가정하지 않는다.

Q31. S0는 무엇인가?

- 짧은 답: 이상적 global FIFO baseline이다.
- 상세 답: Imbalance 하한을 주지만 arbitration/fanout/crossbar 비용과 remote storage를 단순화한다.
- 증거 포인터: paper §V, §VI-B
- 과장 위험: 실제 RTL 구현이나 optimum으로 부르지 않는다.

Q32. S1은 무엇인가?

- 짧은 답: Home cluster local FIFO만 사용하는 static 정책이다.
- 상세 답: Remote movement가 없지만 skew에서 다른 cluster가 idle할 수 있다.
- 증거 포인터: paper §V
- 과장 위험: 모든 조건에서 나쁜 정책이라고 일반화하지 않는다.

Q33. S2는 무엇인가?

- 짧은 답: Idle cluster가 oldest eligible job을 훔치는 정책이다.
- 상세 답: Age를 우선해 balance를 개선하지만 remote traffic이 커질 수 있다.
- 증거 포인터: paper §V
- 과장 위험: Locality-aware라고 부르지 않는다.

Q34. S3는 무엇인가?

- 짧은 답: Age와 locality penalty를 함께 보는 stealing 정책이다.
- 상세 답: Score는 remote weight, activation, reduction owner, bundle mismatch를 반영한 한 design point다.
- 증거 포인터: paper §V
- 과장 위험: 새로운 최적 알고리즘으로 주장하지 않는다.

Q35. Work Stealing이 RTL에 구현됐는가?

- 짧은 답: 제한된 production scheduler/harness 경로에 구현됐다.
- 상세 답: Synthetic jobs 1/5/9가 cycle 16-18에 steal되어 actual MatVec completion까지 identity를 보존했다.
- 증거 포인터: paper §VI-A; RTL tests
- 과장 위험: Gemma 전체가 DDR/link를 거쳐 실행됐다고 말하지 않는다.

Q36. Locality score는 어떻게 쓰는가?

- 짧은 답: Age에서 이동 관련 penalty를 빼 후보를 고른다.
- 상세 답: 양수 threshold와 계수는 calibration되지 않은 design point이며 sensitivity 한계가 있다.
- 증거 포인터: paper §V
- 과장 위험: 보편적 최적값이라고 부르지 않는다.

Q37. S3가 항상 S0보다 좋은가?

- 짧은 답: 아니다.
- 상세 답: S0는 중앙 queue의 이상적 imbalance 하한이고 skew/mixed p95도 S3보다 낮다.
- 증거 포인터: paper §VII-A
- 과장 위험: S0 remote 0 B를 물리 측정으로 사용하지 않는다.

Q38. 왜 S3가 S0보다 느릴 수 있는가?

- 짧은 답: Local queues, eligibility와 locality cost가 선택을 제한하기 때문이다.
- 상세 답: S0는 중앙 arbitration/crossbar 비용을 대부분 생략한 이상 baseline이다.
- 증거 포인터: paper §VI-B
- 과장 위험: 분산 구조의 물리 열세가 증명됐다고 말하지 않는다.

Q39. S3가 balanced에서 이득이 없는 이유는?

- 짧은 답: Queue imbalance가 없어 steal할 필요가 없기 때문이다.
- 상세 답: Balanced p95는 S0-S3 모두 19,524 cycles이고 steals는 0이다.
- 증거 포인터: paper §VII-A
- 과장 위험: 다른 balanced workload까지 동일하다고 일반화하지 않는다.

Q40. Hotspot에서 이득이 없는 이유는?

- 짧은 답: Queue보다 memory channel 병목이 지배하기 때문이다.
- 상세 답: S1과 S3 p95가 159,940 cycles로 같다.
- 증거 포인터: paper §VII-A
- 과장 위험: 실제 DDR hotspot 측정으로 부르지 않는다.

Q41. p95란?

- 짧은 답: 표본의 95%가 그 이하인 지연 경계다.
- 상세 답: 느린 5%가 시작되는 tail 지표로 대화형 지연을 평균보다 잘 드러낸다.
- 증거 포인터: paper §VI-B
- 과장 위험: 단일 run의 worst case와 같다고 말하지 않는다.

Q42. p99를 함께 보는 이유는?

- 짧은 답: 더 극단적인 tail을 확인하기 위해서다.
- 상세 답: S3의 S2 대비 p99 개선은 skew 0.84%, mixed 0.91%로 p95보다 작다.
- 증거 포인터: paper §VII-A
- 과장 위험: 작은 차이를 보드 유의성으로 과장하지 않는다.

Q43. Completion time과 p95의 차이는?

- 짧은 답: P95는 개별 tail 분포, completion은 마지막 작업 종료다.

- 상세 답: S3는 p95를 줄이면서 큰 마지막 job 때문에 completion을 소폭 늘릴 수 있다.
- 증거 포인터: paper §VII-B
- 과장 위험: P95 개선을 전체 makespan 개선으로 등치하지 않는다.

Q44. Compute duty는 무엇인가?

- 짧은 답: 실제 compute-cycle의 전체 cluster-time 비율이다.
- 상세 답: MAC activity에 가까운 정의지만 analytical service model의 값이다.
- 증거 포인터: paper §VI-B
- 과장 위험: 실제 FPGA utilization report로 부르지 않는다.

Q45. Reservation occupancy는 무엇인가?

- 짧은 답: Dispatch부터 대기과 compute 완료까지 예약된 비율이다.
- 상세 답: Link/memory wait가 포함되어 0.99여도 compute가 99% active한 것은 아니다.
- 증거 포인터: paper §VI-B
- 과장 위험: Utilization이라는 축약어만 쓰지 않는다.

Q46. Unreserved idle은 무엇인가?

- 짧은 답: Cluster가 예약되지 않은 cycle 수다.
- 상세 답: Compute-idle과 달리 예약 중 memory/link wait는 포함하지 않는다.
- 증거 포인터: paper §VI-B
- 과장 위험: 전력 idle time으로 직접 사용하지 않는다.

Q47. Full-overlap model은?

- 짧은 답: Link와 memory를 동시에 시작해 max로 data-ready를 둔다.
- 상세 답: Service boundary의 낙관적 가정이며 물리 transaction timing이 아니다.
- 증거 포인터: paper §VI-B
- 과장 위험: 실제 overlap 구현 증거로 말하지 않는다.

Q48. Sequential model은?

- 짧은 답: Link와 memory service를 직렬로 더한다.
- 상세 답: Boundary sensitivity이며 skew에서 S2/S3 순위가 뒤집힐 수 있다.
- 증거 포인터: paper abstract; §VI-B
- 과장 위험: 실제 hardware가 순차라는 증거는 아니다.

Q49. 왜 S3가 S0보다 항상 우수하지 않은가?

- 짧은 답: 정책 목표와 비용 정의가 다르기 때문이다.
- 상세 답: S3는 locality를 지키며 local imbalance를 줄이고, S0는 중앙 공유 비용을 이상화한다.
- 증거 포인터: paper §V-VII
- 과장 위험: S3의 구조적 필요성을 확정하지 않는다.

Q50. Skew에서 S1→S3 p95 변화는?

- 짧은 답: 18.12% 감소다.
- 상세 답: Full-overlap analytical median에서 285,333.05→233,625.15 cycles다.

- 증거 포인터: build/publication_assets/tables/scheduler_core_metrics.csv
- 과장 위험: 보드 latency 감소로 말하지 않는다.

Q51. Mixed에서 S1→S3 p95 변화는?

- 짧은 답: 17.59% 감소다.
- 상세 답: Full-overlap analytical median에서 478,553.05→394,375.45 cycles다.
- 증거 포인터: build/publication_assets/tables/scheduler_core_metrics.csv
- 과장 위험: 다른 workload에 일반화하지 않는다.

Q52. S2→S3 remote weight 변화는?

- 짧은 답: Skew 37.84%, mixed 22.16% 감소다.
- 상세 답: 각각 2,443,776→1,519,104 B, 3,799,040→2,957,312 B다.
- 증거 포인터: paper §VII-B
- 과장 위험: 실제 link bytes로 말하지 않는다.

Q53. S3 completion의 trade-off는?

- 짧은 답: S2보다 skew 0.98%, mixed 0.41% 길다.
- 상세 답: Tail과 traffic 개선이 마지막 completion 개선을 보장하지 않음을 보인다.
- 증거 포인터: paper §VII-B
- 과장 위험: S3가 모든 지표를 개선한다고 말하지 않는다.

Q54. Process repetition 3의 의미는?

- 짧은 답: 동일 seed의 결정성 검사다.
- 상세 답: 독립 표본 수가 아니라 재실행 결과 동일성을 확인한다.
- 증거 포인터: paper §VI-B
- 과장 위험: n=15로 통계 표본을 부풀리지 않는다.

Q55. Seed count는?

- 짧은 답: Synthetic subset에서 5개다.
- 상세 답: Workload별 고정 seed 19, 23, 29, 31, 43의 중앙값을 사용한다.
- 증거 포인터: paper §VI-B
- 과장 위험: Gemma replay도 5 seed라고 말하지 않는다.

Q56. 분석 cycle과 RTL cycle이 같은가?

- 짧은 답: 아니다.
- 상세 답: 분석은 기본 64 MAC/cycle, 현재 RTL은 64 MAC request-to-done 65 cycles로 issue-rate 차이가 65배다.
- 증거 포인터: docs/calibration.md; paper §VI-A
- 과장 위험: 분석 cycle을 FPGA clock에 직접 환산하지 않는다.

Q57. Python과 RTL scheduler semantics가 같은가?

- 짧은 답: 완전히 같지 않다.
- 상세 답: Home 정의, victim 후보, locality cost, dispatch 폭이 다르다.
- 증거 포인터: paper §VI-A 표

- 과장 위험: 동일 구현 교차검증이라고 과장하지 않는다.

Q58. Exact-once 조건은?

- 짧은 답: Input, dispatch, completion ID set이 같고 duplicate가 0이다.
- 상세 답: Drop/duplicate를 숨긴 latency 개선이 아닌지 확인하는 정상성 gate다.
- 증거 포인터: paper §IV, §VI
- 과장 위험: 전체 DMA path exact-once로 확대하지 않는다.

Q59. DRAMsim3는 무엇을 하는가?

- 짧은 답: DDR timing constraint를 cycle 수준에서 모델링한다.
- 상세 답: Channel/bank/row timing을 다루지만 공개 snapshot은 Gemma replay와 결합되지 않았다.
- 증거 포인터: docs/dramsim3.md; paper §II
- 과장 위험: Snapshot을 재현 가능한 end-to-end 결과라 하지 않는다.

Q60. DRAMsim3 결과가 energy에 합산됐는가?

- 짧은 답: 아니다.
- 상세 답: Cycle-by-cycle join이 없어 Gemma J/token에 포함하지 않는다.
- 증거 포인터: paper §II, §VI-C
- 과장 위험: 보존 snapshot을 현재 실행 결과로 말하지 않는다.

Q61. 실제 power를 측정했는가?

- 짧은 답: 아니다.
- 상세 답: Vivado P&R/report_power와 fabricated-board calibrated measurement가 없다.
- 증거 포인터: paper §VI-C; blocked_evidence.csv
- 과장 위험: Energy/token을 measured power로 부르지 않는다.

Q62. Energy/token은 무엇인가?

- 짧은 답: 가정 범위의 dynamic energy 추정치다.
- 상세 답: MAC pJ, link pJ/bit, DRAM dynamic 식을 결합하고 refresh/idle/PHY/board를 제외한다.
- 증거 포인터: paper §VI-C
- 과장 위험: 전체 board energy로 말하지 않는다.

Q63. Link energy sensitivity 범위는?

- 짧은 답: 24/51.2/120 pJ per bit다.
- 상세 답: Low/central/high 가정으로 remote traffic 민감도를 본다.
- 증거 포인터: paper §VI-C
- 과장 위험: 실제 transceiver 측정값으로 말하지 않는다.

Q64. MAC energy sensitivity 범위는?

- 짧은 답: 1/5/15 pJ per INT8 MAC이다.
- 상세 답: Technology-independent design sensitivity이며 implementation power report가 아니다.
- 증거 포인터: paper §VI-C
- 과장 위험: K26 소자의 보장값으로 인용하지 않는다.

Q65. Memory-die-cost normalized metric은?

- 짧은 답: DRAM package 가격을 내부 die 수로 나눈 산술이다.
- 상세 답: Bare-die quote나 전체 BOM이 아니라 capacity/cost sensitivity용 정규화다.
- 증거 포인터: paper §VI-C
- 과장 위험: 구매 가능한 die 가격이라고 말하지 않는다.

Q66. 가격 snapshot의 한계는?

- 짧은 답: 시점·수량·유통사에 따라 변한다.
- 상세 답: 2026-07-31 quantity-1 Mouser/DigiKey snapshot이므로 조달 전 갱신해야 한다.
- 증거 포인터: cost/memory_die_price_snapshot.csv
- 과장 위험: 현재 최저가나 양산가로 보장하지 않는다.

Q67. 왜 K26 local DDR4만 쓰지 않는가?

- 짧은 답: 아직 외부 memory 채택이 확정되지 않았다.
- 상세 답: 먼저 local effective bandwidth, contention, power baseline을 측정해 외부 channel isolation 이득과 비교해야 한다.
- 증거 포인터: paper §IV 표 4
- 과장 위험: Local DDR4가 부족하다고 단정하지 않는다.

Q68. 왜 Gemma 1B가 4 GB에 들어가는데 외부 8 GB인가?

- 짧은 답: 용량 외의 bandwidth/isolation 가설을 검증하기 위한 후보이기 때문이다.
- 상세 답: Context-32K INT8 2.4301 GiB는 nominal 4 GB에 fit하므로 capacity 논거는 기각된다.
- 증거 포인터: paper abstract; §IV
- 과장 위험: 8 GB가 필수라고 말하지 않는다.

Q69. 3B 모델을 실행했는가?

- 짧은 답: 아니다.
- 상세 답: 3B는 generic capacity/model analysis이며 actual end-to-end execution evidence가 없다.
- 증거 포인터: docs/evidence.md; paper §IV
- 과장 위험: Capacity fit을 실행 성공으로 바꾸지 않는다.

Q70. 외부 8 GiB의 물리 구성은?

- 짧은 답: 4 channel×2 GiB다.
- 상세 답: 각 channel은 8 Gb x8 package 두 개로 2 GiB를 구성하는 후보다.
- 증거 포인터: paper §IV
- 과장 위험: Routing 완료 보드 구성으로 말하지 않는다.

Q71. KiCad source는 native인가?

- 짧은 답: 그렇다.
- 상세 답: 텍스트 그림만이 아니라 실제 schematic/PCB source와 render가 있다.
- 증거 포인터: hardware/kicad; paper/final/figures/paper_f07_kicad_coupon_render.png
- 과장 위험: Native source가 곧 제작 준비를 뜻하지 않는다.

Q72. ERC란?

- 짧은 답: Schematic electrical rule check다.
- 상세 답: Pin type, 연결과 선언 규칙을 검사하지만 SI/PI나 실동작은 보장하지 않는다.
- 증거 포인터: scripts/verify_k26_kicad.py
- 과장 위험: ERC pass를 기능 검증으로 부르지 않는다.

Q73. DRC란?

- 짧은 답: PCB design rule check다.
- 상세 답: Clearance/width 등 설정 규칙을 검사하지만 미배선과 제조·신호 무결성 전체를 대신하지 않는다.
- 증거 포인터: scripts/verify_k26_kicad.py
- 과장 위험: 제한 DRC를 fabrication signoff로 부르지 않는다.

Q74. PCB는 fabrication-ready인가?

- 짧은 답: 아니다.
- 상세 답: 55 unrouted nets와 SI/PI/PDN/thermal/EMC 및 FPGA constraint 관문이 남았다.
- 증거 포인터: paper §VIII; study/08_kicad_and_physical_scope.md
- 과장 위험: 제작을 권장하지 않는다.

Q75. KiCad coupon이 증명하는 것은?

- 짧은 답: Native source와 bounded rule-check 가능성이다.
- 상세 답: Physical proposal의 형태와 일부 규칙을 검증하지만 closed hardware datapath는 아니다.
- 증거 포인터: docs/evidence.md
- 과장 위험: 완성 board 기능으로 확대하지 않는다.

Q76. 55 unrouted nets의 의미는?

- 짧은 답: 아직 물리 연결이 완결되지 않았다는 fabrication blocker다.
- 상세 답: 배선, length matching과 후속 DRC가 필요하다.
- 증거 포인터: paper §VIII
- 과장 위험: 단순 시각적 경고로 축소하지 않는다.

Q77. SI/PI/PDN은 각각 무엇인가?

- 짧은 답: Signal integrity, power integrity, power distribution network다.
- 상세 답: 고속 신호 품질과 전원 안정성을 해석·측정하는 물리 검증 단계다.
- 증거 포인터: study/08_kicad_and_physical_scope.md
- 과장 위험: ERC/DRC가 대신한다고 말하지 않는다.

Q78. 다음 물리 검증 순서는?

- 짧은 답: Routing→rule checks→SI/PI/PDN→FPGA timing→fabrication→bring-up→measurement다.
- 상세 답: 각 단계 acceptance criterion을 통과한 뒤 다음으로 간다.
- 증거 포인터: study/08_kicad_and_physical_scope.md
- 과장 위험: 검증 전 제작을 먼저 권하지 않는다.

Q79. 보드에서 가장 먼저 측정할 baseline은?

- 짧은 답: K26 local memory의 동일-workload bandwidth, tail, power다.

- 상세 답: 외부 memory 후보와 같은 trace·정확성 조건으로 비교해야 채택 여부를 결정할 수 있다.
- 증거 포인터: paper §IV, §IX
- 과장 위험: Peak spec만으로 baseline을 대신하지 않는다.

Q80. 실제 link payload bandwidth를 아는가?

- 짧은 답: 아니다.
- 상세 답: GT wrapper, receive, credit/CDC와 payload loop 및 보드 계측이 없다.
- 증거 포인터: docs/architecture.md
- 과장 위험: 4-lane 후보를 measured bandwidth로 바꾸지 않는다.

Q81. 제작 후 성공 기준은?

- 짧은 답: 정확성, exact-once, timing, bandwidth, tail, calibrated power다.
- 상세 답: Local baseline과 동일 workload에서 결과 hash/ID set과 물리 계측을 함께 확인한다.
- 증거 포인터: study/11_whiteboard_explanations.md
- 과장 위험: 부팅 또는 link lock만으로 성공이라 하지 않는다.

Q82. 모델 weights 없이 어떻게 재현하는가?

- 짧은 답: 공개 synthetic/fixture와 manifest-bound 경로를 재현한다.
- 상세 답: Full graph trace는 사용자가 적절한 local artifact를 제공할 때만 hash를 검증해 생성한다.
- 증거 포인터: models/ACQUISITION.md; Makefile
- 과장 위험: Weights가 release에 포함됐다고 말하지 않는다.

Q83. 공식 Source ZIP과 GitHub auto-ZIP 차이는?

- 짧은 답: 공식 ZIP은 source_manifest를 포함하고 auto-ZIP은 fallback tree 검사를 쓴다.
- 상세 답: 둘 다 test/publication-index smoke가 가능하지만 checksum-bound reproduction은 공식 archive가 기준이다.
- 증거 포인터: README.md; scripts/test_source_archive.py
- 과장 위험: Auto-ZIP에 manifest가 있다고 가정하지 않는다.

Q84. Reproduce가 clean tree를 요구하는 이유는?

- 짧은 답: 생성이 추적 파일을 바꾸지 않는지 확인하기 위해서다.
- 상세 답: Git mode에서는 diff와 porcelain, archive mode에서는 manifest/checksum으로 오염을 검사한다.
- 증거 포인터: scripts/verify_clean_source.py; Makefile
- 과장 위험: 단순 명령 성공만 reproducible이라 부르지 않는다.

Q85. 증거 포인터를 답변마다 붙이는 이유는?

- 짧은 답: 청중이 주장을 원본까지 추적할 수 있게 하기 위해서다.
- 상세 답: 수치, 그림, script와 source table을 연결하면 기억 오류와 과장을 줄인다.
- 증거 포인터: release/evidence_index.md
- 과장 위험: 포인터 없는 수치를 새 결과처럼 말하지 않는다.

Q86. 가장 위험한 발표 실수는?

- 짧은 답: 분석 수치를 보드 실측처럼 말하는 것이다.
- 상세 답: 항상 증거 유형과 제외 범위를 수치 앞뒤에 붙인다.

- 증거 포인터: study/09_claim_boundary.md
- 과장 위험: '성능이 향상됐다'만 단독으로 말하지 않는다.

Q87. 질문에 답을 모를 때 어떻게 하는가?

- 짧은 답: 현재 증거로 결정 불가라고 말하고 다음 관문을 제시한다.
- 상세 답: 필요한 실험, 입력, 지표와 acceptance criterion을 구체적으로 답한다.
- 증거 포인터: study/12_interview_script.md
- 과장 위험: 추정값을 즉석 실측처럼 만들지 않는다.

Q88. 이 artifact의 가장 강한 주장과 가장 약한 주장은?

- 짧은 답: 가장 강한 것은 bounded provenance/RTL 기능, 가장 약한 것은 물리 성능 추정이다.
- 상세 답: Graph hash와 parity는 직접/RTL 증거지만 energy와 hybrid latency는 가정 의존적이다.
- 증거 포인터: docs/evidence.md
- 과장 위험: 모든 결과를 같은 confidence로 표현하지 않는다.

Q89. 90초 발표의 마지막 문장은?

- 짧은 답: '어느 경계부터 실제 하드웨어인지'를 함께 보자는 것이다.
- 상세 답: 기여를 evidence-bounded decision artifact로 정리하고 response loop closure를 다음 단계로 제시한다.
- 증거 포인터: presentation/final/outline.md slide 12
- 과장 위험: 완제품 출시 약속으로 끝내지 않는다.

화이트보드 설명

전체 가속기 — 90초

왼쪽에 K26의 네 compute cluster, 오른쪽에 4-channel Memory FPGA를 그린다. TileJob에는 계산 범위와 home/locality 정보가 있다. 현재 compute RTL은 scheduler에서 payload store와 MatVec까지 닫혀 있지만 DDR response와 link receive는 닫히지 않았다. 따라서 결과는 정책 분석과 제한된 RTL 증거이며 보드 가속 성능이 아니다.

Work Stealing — 60초

두 local queue를 그린다. 한쪽이 비고 다른 쪽이 길면 idle cluster가 victim을 찾는다. S2는 가장 오래된 eligible job, S3는 age에서 locality penalty를 뺀 score가 좋은 job을 고른다. tail은 줄 수 있지만 remote bytes와 마지막 completion이 나빠질 수 있다.

메모리 계층 — 60초

모델 weight, activation, KV cache를 분리한다. Channel은 독립 대역폭, bank는 내부 병렬성, row hit는 activation 비용 감소와 관계가 있다. Capacity fit과 bandwidth adequacy는 다른 질문이다.

p95가 중요한 이유 — 45초

평균이 같아도 일부 요청이 매우 느리면 대화형 체감이 나쁘다. p95는 100개 중 느린 5개가 시작되는 경계다. 단 p95만 최적화하면 completion이나 traffic이 악화될 수 있어 함께 본다.

왜 완성 가속기가 아닌가 — 45초

compute, command, routing plane은 있지만 실제 weight가 DDR에서 응답되어 link를 건너 MatVec payload store에 들어오는 페루프가 없다. 이 경계를 공개하는 것이 연구 신뢰성의 일부다.

다음 하드웨어 검증 — 60초

DMA request와 DDR response를 연결하고 credit/CDC가 있는 link receive를 만든다. returned weight를 FIFO/payload store에 넣어 exact-once MatVec completion을 확인한다. 그 뒤 local DDR4 baseline과 동일 workload로 bandwidth, p95, board power를 계측한다.

면접·심사 답변 스크립트

20초 소개

“저는 Gemma 3 1B 실제 graph에서 TileJob ledger를 만들고, K26 compute cluster의 정적 queue imbalance를 locality-aware Work Stealing으로 완화할 조건을 분석했습니다. 결과는 analytical model과 제한된 RTL/KiCad evidence이며 보드 성능으로 과장하지 않았습니다.”

수치 답변 공식

“이 수치는 [증거 유형]에서 [조건]으로 얻었습니다. 비교값은 [baseline]이고 변화는 [수치]입니다. 다만 [제외 범위]를 포함하지 않으므로 [금지 주장]은 하지 않습니다.”

모르는 질문

“현재 증거로는 결정할 수 없습니다. 필요한 다음 실험은 [관문]이고, 통과 기준은 [측정 지표/acceptance criterion]입니다.”

셀프 퀴즈

- 1. Decode와 prefill의 memory access 차이를 30초 안에 설명하라.
- 2. projection/token이 183인 근거 유형은 무엇인가?
- 3. S0, S1, S2, S3를 한 문장씩 비교하라.
- 4. 왜 reservation occupancy를 utilization이라고 부르면 위험한가?
- 5. Full-overlap과 sequential의 식을 써라.
- 6. skew/mixed p95 감소율과 증거 유형을 말하라.
- 7. S3가 S2보다 나빠지는 지표 하나를 말하라.
- 8. 외부 8 GiB가 capacity로 필수 아닌 이유를 말하라.
- 9. RTL로 달린 데이터 경로와 열린 경로를 그려라.
- 10. KiCad ERC/DRC가 보장하지 않는 것 세 가지를 말하라.
- 11. 실제 power 측정인지 묻는 질문에 20초로 답하라.
- 12. 다음 hardware validation을 순서대로 말하라.

정답은 00, 05-10에서 근거 문장과 함께 확인한다.

흔한 오해

오해 · 바로잡기

메모리가 병목이므로 외부 FPGA가 반드시 필요하다 · 로컬 DDR4 baseline이 없어 채택은 보류 상태다

S3는 항상 가장 빠르다 · balanced/hotspot 이득이 없고 sequential skew에서 순위가 바뀐다

occupancy 99%는 MAC 활용률 99%다 · 대기 포함 예약률이며 compute duty와 다르다

S0 remote bytes 0은 측정 결과다 · 중앙 공유 storage를 가정한 모델 정의다

3/3 tile parity는 full model 정확성이다 · 세 representative 16×4 tile의 제한 증거다

ORT 408.445 ms는 accelerator 결과다 · Y700 Android CPU EP 기능 참조다

DRAMsim3 snapshot이 Gemma replay에 결합됐다 · 공개 snapshot은 보존 자료이며 재생성·결합되지 않았다

ERC/DRC pass면 제작 가능하다 · unrouted/SI/PI/PDN/thermal 등 관문이 남는다

8 GB는 3B 실행을 증명한다 · generic capacity arithmetic일 뿐 실행 증거가 아니다

VARP 발표·논문 방어 스터디팩

이 자료는 논문을 외우기보다 주장-증거-한계를 연결해 설명하도록 돕는다. 권장 순서는 00 → 01 → 02 → 03 → 10 → 13이다.

30분 학습 경로

- 1. 한 장 요약과 개념 지도를 읽는다.
- 2. 슬라이드별 핵심 문장을 소리 내어 말한다.
- 3. Q&A 1-20번과 적대적 질문을 답한다.
- 4. 답변마다 증거 유형을 먼저 밝힌다.

답변 규칙

- 수치 앞에 분석 모델, RTL 시뮬레이션, 그래프 유도, 호스트 측정 중 하나를 붙인다.
- 구현된 compute path와 미통합 DDR/link response path를 구분한다.
- 모르면 추측하지 말고 다음 검증 관문을 말한다.